

GUÍA PARA PERSONAS NUEVAS

ARKA-Lite desde cero

Instala lo necesario, obtiene acceso al repositorio y ejecuta los dos servicios por primera vez en Windows.

Ruta principal: Windows + PowerShell + Docker Compose

No necesitas saber Java, Spring Boot ni Kubernetes para completar la primera prueba. Los pasos opcionales están al final.

Meta del recorrido: abrir <http://localhost:8080/solicitudes>, enviar **INC-002** y encontrar su aviso en <http://localhost:8081/notificaciones>.

Edición de septiembre de 2026 · Basada en los archivos vigentes de ARKA-Lite

Antes de empezar

Esta guía supone una PC Windows con conexión a internet y permiso para instalar aplicaciones. Usa **PowerShell de Windows**; cuando un paso requiera abrirlo como administrador, lo diremos. Conserva la terminal abierta durante cada comando que deje un proceso en ejecución.

Herramienta	Para qué se usa	¿Necesaria en la primera prueba?
Temurin 25 y 26	Aprender y desarrollar Java; ARKA-Lite compila con 26.	No para Compose; sí para Maven e IntelliJ.
IntelliJ IDEA	Leer, editar y depurar el código.	No para Compose.
Git + GitHub	Descargar el proyecto y colaborar.	Sí.
WSL 2 + Docker Desktop	Ejecutar los dos servicios en contenedores Linux.	Sí.

Mapa de la guía

1

Java e IntelliJ

Instala los JDK y el editor; deja Java 26 activo.

2

Git y WSL

Configura tu identidad Git y prepara el motor Linux de Docker.

3

Docker y GitHub

Instala Docker Desktop, consigue acceso y clona el repositorio.

4

Primera ejecución

Arranca con Compose y verifica que los dos servicios respondan.

5

Siguientes pasos

Aprende el flujo de trabajo y, si lo necesitas, Kubernetes.

No instales Maven por separado. Cada microservicio incluye su Maven Wrapper. Tampoco ejecutes el **pom.xml** de la raíz: corresponde al monolito anterior.

1 · Instala Java 25, Java 26 e IntelliJ

ARCA-Lite usa **Java 26**. Instala Temurin 25 si el equipo lo solicita, pero configura la terminal e IntelliJ con **Temurin 26** para este proyecto.

Abre [Temurin 25](#) y [Temurin 26](#). En cada página elige **Windows** → **tu arquitectura** → **JDK** → **MSI**, descarga e instala. Acepta las opciones que añaden Java al sistema si aparecen. La [guía oficial del instalador](#) explica las pantallas.

Después abre una **nueva** ventana PowerShell y verifica:

```
java --version
javac --version
$env:JAVA_HOME
where.exe java
```

Debe salir 26 en los dos primeros comandos. Si aparece 25 u otra versión, cambia la variable de entorno **JAVA_HOME** a la carpeta instalada del JDK 26 y pon su subcarpeta **bin** antes de otras versiones en **Path**; cierra y reabre PowerShell.

Instala [IntelliJ IDEA](#). Aún no necesitas abrir un proyecto. Cuando clones ARCA-Lite, importarás **servicio-solicitudes/pom.xml** y **servicio-notificaciones/pom.xml** como proyectos Maven independientes.

Comprobación: `java --version` y `javac --version` indican 26. Si solo usarás Docker Compose hoy, puedes continuar aunque el JDK aún no esté configurado.

2 · Instala Git y prepara WSL 2

Git: identidad para tus cambios

Descarga [Git para Windows](#). Abre otra ventana PowerShell y ejecuta; sustituye nombre y correo por los tuyos:

```
git --version
git config --global user.name "Nombre Apellido"
git config --global user.email "tu-correo@example.com"
git config --global --get user.name
git config --global --get user.email
```

Git guarda estos datos para los commits en tu PC. No crea tu cuenta GitHub ni concede acceso al proyecto.

WSL 2: base de Docker Desktop

Sigue la [guía de Microsoft para WSL](#). Si aún no lo tienes, abre **PowerShell como administrador** y ejecuta:

```
wsl --install
```

Reinicia cuando Windows lo pida. En la primera apertura de Ubuntu, crea el usuario Linux que solicite el asistente. Luego, en PowerShell de Windows:

```
wsl --version
wsl --status
wsl --list --verbose
```

La distribución debe indicar versión **2**. Si ya tenías WSL, usa **wsl --update** y revisa la guía de Microsoft ante errores de virtualización.

Importante: los comandos de esta guía se escriben en PowerShell de Windows, no dentro de Ubuntu. WSL permite que Docker ejecute contenedores Linux.

3 · Cuenta Docker e instalación

Crea tu cuenta Docker antes de instalar la aplicación. Es una cuenta distinta de GitHub y de la cuenta de Windows. Si tu organización ya te asignó una, usa esa cuenta.

A. Crea y confirma la cuenta Docker

1. Abre la página oficial de registro. Elige correo y contraseña, o continúa con Google/GitHub si esa es tu cuenta de trabajo.
2. Elige tu **Docker ID** (nombre de usuario). Comprueba bien el nombre antes de guardar: Docker indica que no se puede cambiar después.
3. Si te registras con correo, escribe el código que Docker te envía. Luego inicia sesión y comprueba que ves tu perfil.

[Abrir registro y ayuda oficial de Docker](#)

B. Instala Docker Desktop en Windows

4. Descarga **Docker Desktop para Windows** desde la página oficial. Ejecuta el instalador.
5. Cuando pregunte por el motor, selecciona **WSL 2** y contenedores Linux. Sigue el asistente y reinicia si te lo pide.
6. Abre Docker Desktop. Acepta sus condiciones, inicia sesión con tu cuenta Docker si se solicita y espera a que el motor indique que está listo.

[Descargar Docker Desktop para Windows](#)

C. Comprueba que funciona

Abre una PowerShell **nueva** y ejecuta cada línea. El último comando descarga una imagen pequeña y muestra un mensaje de bienvenida.

```
docker version
docker compose version
docker run --rm hello-world
```

Si falla: Deja Docker Desktop abierto. Si `docker version` muestra solo el cliente, espera al motor. Si hay un error de WSL, vuelve a la página anterior y verifica `wsl --status`.

4 · Cuenta GitHub y clonación

Necesitas una cuenta GitHub para acceder al código. Tener Git instalado en la PC no crea esa cuenta ni te da permiso en el repositorio.

A. Crea tu cuenta GitHub

1. Abre **github.com/signup** y sigue el formulario: correo, contraseña y nombre de usuario.
2. Confirma el correo desde el mensaje de GitHub. Entra a GitHub y ubica tu nombre de usuario en el perfil.
3. Comparte **solo tu nombre de usuario** con la persona responsable de ARKA-Lite y pide acceso al repositorio. Acepta la invitación cuando llegue.

[Crear cuenta GitHub](#)

[Abrir repositorio ARKA-Lite](#)

Comprueba el acceso: Abre el enlace del repositorio mientras tienes iniciada la sesión correcta. Si no puedes verlo o colaborar, espera la invitación antes de seguir.

B. Clona el código en tu PC

4. Abre PowerShell. Ve a la carpeta donde guardas proyectos; el ejemplo crea una dentro de Documentos.
5. Copia las líneas siguientes **una por una**. `git clone` crea la carpeta `arka-lite` con el código; `cd` entra a ella.

```
$proyectos = Join-Path $HOME 'Documents\Proyectos'  
New-Item -ItemType Directory -Force -Path $proyectos  
Set-Location $proyectos  
git clone https://github.com/jfranceschi-dev/arka-lite.git  
cd arka-lite  
git remote -v  
git status
```

Resultado esperado: `git remote -v` muestra la dirección de ARKA-Lite y `git status` indica que estás en un repositorio. Dentro deben existir `compose.yaml`, `servicio-solicitudes/` y `servicio-notificaciones/`.

Si no puedes clonar: Comprueba que aceptaste la invitación y que GitHub abrió la cuenta correcta. Si aparece una ventana de credenciales, inicia sesión allí. Un error 404 también puede significar falta de acceso.

5 · Enciende los dos servicios

Abre Docker Desktop. En PowerShell, asegúrate de estar en la raíz de `arka-lite`, donde está `compose.yaml`.

```
docker compose config
docker compose up --build -d
docker compose ps
```

La primera construcción puede tardar varios minutos: descarga imágenes base y dependencias Java. Espera a que `docker compose ps` muestre los dos servicios iniciados; notificaciones debería aparecer saludable. Si falla, lee los registros:

```
docker compose logs servicio-notificaciones
docker compose logs servicio-solicitudes
```

Puedes abrir en el navegador:

- Solicitudes: <http://localhost:8080/swagger-ui/index.html>
- Notificaciones: <http://localhost:8081/swagger-ui/index.html>

El puerto 8081 de Windows lleva al puerto 8080 del contenedor de notificaciones. Ambos programas Java escuchan dentro de sus contenedores en 8080; Compose les da nombres y una red privada.

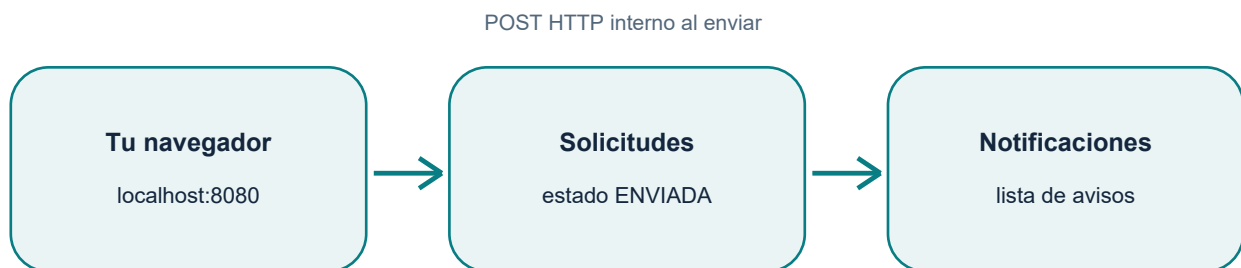
Comprobación: `docker compose ps` muestra los dos contenedores. Si un puerto está ocupado, consulta la sección de problemas frecuentes antes de cambiarlo.

6 · Haz la primera prueba completa

Con ambos contenedores encendidos, ejecuta estas órdenes en PowerShell desde cualquier carpeta. Las respuestas aparecerán como datos en la terminal:

```
Invoke-RestMethod http://localhost:8080/solicitudes
Invoke-RestMethod -Method Post http://localhost:8080/solicitudes/crear
Invoke-RestMethod -Method Post http://localhost:8080/solicitudes/INC-002/enviar
Invoke-RestMethod http://localhost:8081/notificaciones
```

La segunda orden crea **INC-002** en estado **BORRADOR**. La tercera lo cambia a **ENVIADA**. La última debe mostrar una notificación de ese envío. Al arrancar, la aplicación ya carga INC-001 y CAM-002 y envía CAM-002; por eso puedes ver otros registros.



La llamada desde solicitudes a notificaciones usa HTTP dentro de la red de Compose. Si notificaciones falla, solicitudes conserva **ENVIADA** y registra un error; no hay reintento automático.

Apaga el laboratorio

```
docker compose down
```

Los datos se guardan solo en memoria: al apagar y volver a iniciar los servicios, se pierden los cambios anteriores. Los datos de ejemplo se crearán nuevamente al arrancar.

Si algo no funciona

Síntoma	Qué revisar
java --version no indica 26	Revisa JAVA_HOME, Path y abre una terminal nueva. Para Compose puedes seguir sin Java local.
Docker no se conecta al servidor	Abre Docker Desktop; confirma WSL 2 y espera a que el motor termine de iniciar.
No puedes clonar	Verifica la URL, la cuenta GitHub y la invitación de acceso.
8080 u 8081 están ocupados	Usa <code>Get-NetTCPConnection -LocalPort 8080,8081 -ErrorAction SilentlyContinue</code> ; cierra el proceso anterior o cambia el puerto del host en Compose.
Un contenedor se detiene	Ejecuta <code>docker compose ps -a</code> y <code>docker compose logs servicio-solicitudes</code> desde la raíz.
No aparece la notificación	Revisa ambos logs. La solicitud puede quedar ENVIADA aunque el aviso HTTP falle.
Cambiaste código y no se refleja	Ejecuta <code>docker compose up --build -d</code> para reconstruir la imagen.

Para pedir ayuda: comparte el paso exacto, el comando ejecutado y el mensaje de error. Evita compartir contraseñas, tokens y claves privadas.

Después de la primera ejecución

Si vas a desarrollar, abre la raíz en IntelliJ e importa los dos `pom.xml` de servicio. Confirma JDK 26 y ejecuta las pruebas de cada servicio por separado:

```
cd servicio-solicitudes
.\mvnw.cmd clean verify
cd ../servicio-notificaciones
.\mvnw.cmd clean verify
```

Si vas a colaborar, lee el flujo de ramas, commits y Pull Requests en [docs/07-desarrollo/](#). Antes de modificar la aplicación, consulta [docs/04-proyecto/](#) y los diagramas en [docs/05-arquitectura/](#).

Kubernetes: paso posterior y opcional

Cuando Compose funcione, puedes crear el clúster **kind integrado en Docker Desktop**, instalar la CLI `kind`, construir y cargar las imágenes `:local`, aplicar `k8s/` y abrir dos `port-forward`. El procedimiento completo está en [docs/06-uso/04-kubernetes-local.md](#). No ejecutes `kind create cluster` para esta ruta.

K9s, HTTPS y certificados también son opciones posteriores; sus guías están en [docs/06-uso/](#). No forman parte de la primera ejecución.

Lista final

- [] Java 26 confirmado para desarrollo; Git, WSL 2 y Docker disponibles.
- [] Cuenta GitHub con acceso y repositorio clonado.
- [] Ambos contenedores iniciados con Compose.
- [] INC-002 enviado y su notificación visible.
- [] Laboratorio apagado con `docker compose down`.

Enlaces oficiales y material del proyecto

Temurin 25/26: adoptium.net/temurin/releases/ · IntelliJ: jetbrains.com/idea/download/ · Git: git-scm.com/install/windows/ · WSL: learn.microsoft.com/es-es/windows/wsl/install · Docker Desktop: docs.docker.com/desktop/setup/install/windows-install/ · GitHub: github.com/jfranceschi-dev/arka-lite

Documentación ampliada: `docs/README.md` dentro del repositorio. Los comandos y los puertos de este PDF se contrastaron con `compose.yaml`, los `pom.xml` y el código de ambos servicios.

K9s para Kubernetes (opcional)

K9s es opcional. Sirve para explorar el clúster Kubernetes desde una terminal. **No hace falta para la primera ejecución con Docker Compose.** Úsalo cuando ya hayas completado la guía `docs/06-uso/04-kubernetes-local.md`.

A. Prepara Kubernetes

1. En Docker Desktop, abre **Kubernetes** → **Create cluster**, elige **kind** y espera a que el clúster termine de iniciar.
2. Abre PowerShell y confirma que el contexto actual sea `docker-desktop` y que haya un nodo `Ready`. Para ver ARKA-Lite, aplica antes los manifiestos de `k8s/` como indica la guía del proyecto.

```
kubectl config current-context
kubectl get nodes
kubectl get pods
```

Importante: K9s actúa sobre el contexto Kubernetes activo. Si el contexto no dice `docker-desktop`, no continúes hasta corregirlo.

B. Instala y abre K9s

3. En PowerShell de Windows, instala K9s con el administrador de paquetes de Windows. Confirma las preguntas del instalador. Abre **otra** PowerShell y verifica la versión antes de entrar.

[Métodos oficiales de instalación de K9s](#)

```
winget install k9s
k9s version
k9s
```

C. Revisa ARKA-Lite dentro de K9s

4. Escribe `:pods` y Enter. Debes ver los pods de solicitudes y notificaciones en el namespace `default`; revisa que estén listos.
5. Selecciona un pod con las flechas y pulsa `l` para ver sus logs. Pulsa `Esc` para volver y `d` para ver detalles.
6. Escribe `:deploy` para Deployments o `:svc` para Services. Pulsa `?` para ver los atajos de tu versión y escribe `:quit` para salir.

Precaución: K9s también permite modificar o borrar recursos. Para cambios compartidos por el equipo, edita los manifiestos de `k8s/` y aplícalos con `kubectl`.